

Lab 5: Les Modules

Durée: 20 minutes

Les fichiers de configuration peuvent être séparés en modules pour mieux organiser votre configuration. Cela rend votre code plus facile à lire et réutilisable dans toute votre organisation. Vous pouvez également utiliser le registre public de modules pour rechercher des modules préconfigurés.

- Tâche 1: Refactoriser votre code existant dans un module local
- Tâche 2: Explorez le registre Pubic des modules et installez un module
- Tâche 3: Actualisez et réexécutez votre configuration Terraform

Tâche 1: Refactoriser votre code existant dans un module local

Un module Terraform n'est qu'un ensemble de configuration. Pour ce lab, nous allons refactoriser votre configuration existante afin que la configuration du serveur Web soit à l'intérieur d'un module.

Étape 1.1

Créez un nouveau répertoire appelé `server` dans votre répertoire `terraform` et créez un nouveau fichier à l'intérieur appelé `server.tf`.

Étape 1.2

Editez le fichier `server/server.tf`, avec le contenu suivant:

```
variable ami {}
variable subnet_id {}
variable vpc_security_group_ids {
  type = list
}
variable identity {}

resource "aws_instance" "web" {
  ami                = var.ami
  instance_type      = "t2.micro"
  subnet_id          = var.subnet_id
  vpc_security_group_ids = var.vpc_security_group_ids

  tags = {
    "Identity"    = var.identity
    "Name"        = "Student"
    "Environment" = "Training"
  }
}
```

```
output "public_ip" {
  value = aws_instance.web.public_ip
}

output "public_dns" {
  value = aws_instance.web.public_dns
}
```

Étape 1.3

Dans votre configuration racine (également appelée le module racine) `terraform/main.tf`, nous pouvons supprimer les références précédentes à votre configuration et les refactoriser en tant que module.

```
# Remove the entire block:
#
# resource "aws_instance" "web" {
# ...
# }

module "server" {
  source = "./server"

  ami           = var.ami
  subnet_id     = var.subnet_id
  vpc_security_group_ids = var.vpc_security_group_ids
  identity      = var.identity
}
```

Nos 'outputs' doivent également être définies dans le module racine. Au bas de votre configuration `main.tf`, modifiez les outputs public IP et public DNS pour correspondre au code suivant. Notez la différence d'interpolation maintenant que les informations sont fournies par un module.

```
output "public_ip" {
  value = module.server.public_ip
}

output "public_dns" {
  value = module.server.public_dns
}
```

Étape 1.4

Maintenant, lancez `terraform get` ou `terraform init` pour installer le module. Puisque nous ajoutons simplement un module et aucun fournisseur, `get` est suffisante, mais `init` est également 'safe' à utiliser. Même les modules locaux doivent être installés avant de pouvoir être utilisés.

Une fois que vous avez fait cela, vous pouvez exécuter à nouveau `terraform apply`. Notez que l'instance sera recréée et son identifiant modifié, mais tout le reste doit rester le même.

```
terraform apply
```

```
# aws_instance.web will be destroyed
- resource "aws_instance" "web" {

...

# module.server.aws_instance.web will be created
+ resource "aws_instance" "web" {

...

```

Tâche 2: Explorer le registre public des modules

Étape 2.1

HashiCorp héberge un registre de modules à l'adresse: <https://registry.terraform.io/>

Le registre contient un grand ensemble de modules fournis par la communauté que vous pouvez utiliser dans vos propres configurations. Explorez le registre pour voir ce qui est disponible.

Étape 2.2

Recherchez "dynamic-keys" dans le registre public et décochez la case "Verified". Vous devriez alors voir un module appelé "dynamic-keys" créé par l'un des fondateurs de HashiCorp, Mitchell Hashimoto. Vous pouvez également accéder directement à <https://registry.terraform.io/modules/mitchellh/dynamic-keys/aws/2.0.0>.

Sélectionnez ce module et lisez le contenu des onglets [Readme](#), [Inputs](#), [Outputs](#), et [Resources](#). Ce module générera une paire de clés publique et privée afin que vous puissiez SSH dans votre instance.

Étape 2.3

Pour intégrer ce module dans votre configuration, ajoutez-le après votre `provider block` dans `main.tf`:

```
module "keypair" {
  source = "mitchellh/dynamic-keys/aws"
  version = "2.0.0"
  path    = "${path.root}/keys"
  name    = "${var.identity}-key"
}
```

**** __ Ce module expose les informations de clé privée dans Terraform state et ne doit pas être utilisé en production! __ ****

Vous faites maintenant référence au module, mais Terraform devra télécharger la source du module avant de l'utiliser. Exécutez la commande `terraform init` pour le télécharger.

Pour provisionner les ressources définies par le module, exécutez `terraform apply`, et répondez `yes` à l'invite de confirmation.

Étape 2.4

Nous allons maintenant utiliser le module *keypair* pour installer une clé publique sur notre serveur. Dans `main.tf`, ajoutez les outputs nécessaires de notre module *keypair* à notre module *server*:

```
module "server" {
  source = "./server"

  ami            = var.ami
  subnet_id      = var.subnet_id
  vpc_security_group_ids = var.vpc_security_group_ids
  identity       = var.identity
  key_name       = module.keypair.key_name
  private_key    = module.keypair.private_key_pem
}
```

Étape 2.5

Dans votre fichier `server/server.tf`, ajoutez deux nouvelles variables au reste des variables en haut du fichier:

```
variable key_name {}
variable private_key {}
```

Ajoutez la variable *key_name* au bloc de ressources *aws_instance* dans `server/server.tf`:

```
resource "aws_instance" "web" {
  ami            = var.ami
  instance_type  = "t2.micro"
  subnet_id      = var.subnet_id
  vpc_security_group_ids = var.vpc_security_group_ids
  key_name       = var.key_name

  # ... leave the rest of the block unchanged
}
```

Nous utiliserons la variable *private_key* plus tard.

Tâche 3: Actualisez et réexécutez votre configuration Terraform

Étape 3.1

Réexécutez `terraform apply` pour supprimer l'instance d'origine et la recréer à nouveau. La clé publique sera maintenant installée sur la nouvelle instance.

La destruction de l'ancienne instance et la mise en place de la nouvelle peuvent prendre quelques minutes. Vous remarquerez peut-être que ces deux choses se produisent en parallèle:

```
...  
  
aws_instance.web: Destroying... [id=i-00b20b227c41eca94]  
module.server.aws_instance.web: Creating...  
aws_instance.web: Still destroying... [id=i-00b20b227c41eca94, 10s elapsed]  
module.server.aws_instance.web: Still creating... [10s elapsed]  
  
...
```

Puisqu'il n'y a aucune dépendance entre les deux, terraform peut effectuer les deux opérations en même temps. Cela signifie que pendant que l'application est toujours en cours d'exécution, les deux instances peuvent exister en même temps.

Vous verrez également que les sorties incluent désormais une liste (pour l'instant) des valeurs *public_dns* et *public_ip*:

```
...  
  
Apply complete! Resources: 1 added, 0 changed, 1 destroyed.  
  
Outputs:  
  
public_dns = ec2-54-184-51-90.us-west-2.compute.amazonaws.com  
public_ip  = 54.184.51.90
```

Lorsque nous avons déplacé la configuration des outputs vers le module *server*, nous avons changé ces outputs en des listes. Ceci afin que nous puissions mettre à jour le module pour créer plusieurs instances à la fois dans un futur lab.